

Interesting Pairs Count

Absolutely. Let's understand this problem **from the meaning first**, then derive the formula naturally.

1. What is the question actually saying?

You are given an array:

```
a = [6, 2, 3, 8, 1]
```

Hossam chooses one element a_i , and Hazem chooses another element a_j .

They form a pair (a_i, a_j) .

There are 3 important conditions:

1. They must choose **different indices**:

```
i != j
```

2. We calculate their absolute difference:

```
|a_i - a_j|
```

3. This difference must be the **maximum possible difference in the entire array**.

2. What is the maximum possible difference?

Look at:

```
[6, 2, 3, 8, 1]
```

The smallest value is:

```
min = 1
```

The largest value is:

```
max = 8
```

The largest possible absolute difference is therefore:

```
|8 - 1| = 7
```

Why?

Because no two numbers can have a difference greater than:

```
maximum - minimum
```

So the problem is basically asking:

How many ordered pairs (i, j) have one element equal to the minimum and the other equal to the maximum?

That's the key observation.

3. Why only minimum and maximum matter?

Suppose:

```
a = [6, 2, 3, 8, 1]
```

Maximum difference = $8 - 1 = 7$.

Can 6 participate?

```
|6 - 1| = 5
|8 - 6| = 2
```

Neither is 7.

Can 3 participate?

```
|3 - 1| = 2
|8 - 3| = 5
```

Again, not 7.

Only:

```
8 and 1
```

can produce the maximum difference.

So we don't actually care about all pairs.

We only care about:

- how many times the **minimum** occurs
- how many times the **maximum** occurs

4. Example 1

```
a = [6, 2, 3, 8, 1]
```

Minimum:

```
1
```

It occurs:

```
1 time
```

Maximum:

```
8
```

It occurs:

```
1 time
```

Therefore:

```
number of pairs = 1 × 1 × 2
                  = 2
```

Why multiply by 2?

Because the pair is **ordered**.

We can choose:

```
Hossam = 8
Hazem  = 1
```

or:

```
Hossam = 1
Hazem  = 8
```

So:

```
(8, 1)
(1, 8)
```

Answer:

```
2
```

5. Example 2

Consider:

```
a = [7, 2, 8, 3, 2, 10]
```

Minimum:

```
2
```

How many times?

```
2 → 2 times
```

Maximum:

```
10
```

How many times?

```
10 → 1 time
```

Maximum difference:

```
10 - 2 = 8
```

Now every 2 can pair with the 10.

We have:

```
21 → 10
22 → 10
```

That's 2 pairs.

But remember, order matters:

```
2 → 10
10 → 2
```

So:

```
2 × 1 × 2 = 4
```

Answer:

```
4
```

6. The important intuition

The entire problem can be reduced to this:

Find the minimum and maximum values. Count how many times each occurs.

Let:

```
cntMin = number of occurrences of minimum
cntMax = number of occurrences of maximum
```

Then:

```
answer = cntMin × cntMax × 2
```

Why?

For every occurrence of minimum, we can choose any occurrence of maximum.

Therefore:

```
cntMin × cntMax
```

pairs.

And because:

```
(min, max)
```

and

```
(max, min)
```

are different ordered pairs:

```
answer = cntMin × cntMax × 2
```

7. One important special case

What if:

```
a = [5, 5, 5, 5]
```

Then:

```
min = 5
max = 5
```

The maximum absolute difference is:

```
|5 - 5| = 0
```

Now **every two different indices** form a valid pair.

There are n choices for the first index and $n-1$ choices for the second:

```
answer = n * (n - 1)
```

For $n = 4$:

```
4 * 3 = 12
```

So this case needs separate handling.

8. Final approach

For every test case:

Step 1

Find:

```
mn = minimum element
mx = maximum element
```

Step 2

Count:

```
cntMin = frequency of mn
cntMax = frequency of mx
```

Step 3

If:

```
mn == mx
```

then:

```
answer = n * (n - 1)
```

Otherwise:

```
answer = cntMin * cntMax * 2
```

9. Complexity

We scan the array once.

```
Time: O(n)
Space: O(1)
```

Since the total n over all test cases is at most 10^5 , this is easily fast enough.

10. C++ implementation

```
C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        int n;
        cin >> n;

        vector<long long> a(n);

        long long mn = LLONG_MAX;
        long long mx = LLONG_MIN;

        for (int i = 0; i < n; i++) {
            cin >> a[i];

            mn = min(mn, a[i]);
            mx = max(mx, a[i]);
        }

        long long cntMin = 0;
        long long cntMax = 0;

        for (int i = 0; i < n; i++) {
            if (a[i] == mn)
                cntMin++;

            if (a[i] == mx)
                cntMax++;
        }

        if (mn == mx) {
            cout << 1LL * n * (n - 1) << '\n';
        }
        else {
            cout << 2LL * cntMin * cntMax << '\n';
        }
    }

    return 0;
}
```

The core thing to remember

Don't think about **all n^2 pairs**.

Think:

```
Maximum absolute difference
    ↓
max - min
    ↓
Only min and max can produce it
    ↓
Count min × count max
    ↓
Multiply by 2 because pairs are ordered
```

So the whole problem is essentially a **frequency-counting problem disguised as a pair problem**.

Good question. The important thing is: **`min == max` does not merely mean “the minimum and maximum values are the same.” In an array, that can only happen if every element is the same.**

Why?

Suppose the array is:

```
[5, 5, 5, 5]
```

Then:

```
min = 5
max = 5
```

Now imagine there was some different value.

For example:

```
[5, 5, 7, 5]
```

Then:

```
min = 5
max = 7
```

So they would **not** be equal.

Another example:

```
[5, 5, 3, 5]
```

Then:

```
min = 3
max = 5
```

Again, not equal.

Therefore:

```
min == max
```

can happen **only when every element has exactly that same value**.

Think of it mathematically

For every element $a[i]$, we always have:

```
min <= a[i] <= max
```

If:

```
min == max
```

then:

```
min <= a[i] <= min
```

The only possibility is:

```
a[i] = min
```

for **every** i .

So:

```
min == max
      ↓
every element is equal
      ↓
maximum difference = 0
      ↓
every pair of different indices is valid
```

Why $n \times (n-1)$?

For:

```
[5, 5, 5, 5]
```

Choose Hossam's index:

```
4 choices
```

Then choose Hazem's index:

```
3 remaining choices
```

So:

```
4 × 3 = 12
```

And we **do not use** $cntMin \times cntMax \times 2$ here because min and max represent the **same value**. Multiplying by 2 would count the same index-pairs incorrectly.

This is exactly why the code has:

```
C++
if (mn == mx)
    ans = 1LL * n * (n - 1);
else
    ans = 2LL * cntMin * cntMax;
```

ChatGPT can make mistakes. Check important info.